

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERIA

ESCUELA DE CIENCIAS Y SISTEMAS

ORGANIZACION DE LENGUAJES Y COMPILADORES 2

SECCIÓN A

MANUAL TECNICO

NOMBRE	CARNÉ
VICTOR HUGO VELASQUEZ	202100054
BRANDON EDUARDO PABLO GARCIA	202112092

Guatemala, junio del 2026

INTRODUCCION

Bienvenido al manual técnico de CreditGuard, un sistema inteligente diseñado para la evaluación de riesgo crediticio mediante técnicas de aprendizaje supervisado. Este documento está dirigido a desarrolladores, analistas de datos, ingenieros de machine learning y profesionales del sector financiero que deseen comprender a fondo la arquitectura, funcionamiento e integración técnica de CreditGuard.

A lo largo de este manual, se describen los fundamentos del sistema, incluyendo la construcción del motor de predicción, el preprocesamiento de variables clave, la integración con FastAPI para la exposición de servicios predictivos, y las estrategias implementadas para garantizar la precisión y confiabilidad de las evaluaciones. CreditGuard utiliza un conjunto de variables críticas —como nivel de ingresos, historial de pagos, nivel de endeudamiento, antigüedad laboral y comportamiento crediticio previo— para determinar si un solicitante presenta o no riesgo de incumplimiento en el pago de un crédito.

El objetivo de este manual es proporcionar una guía clara y estructurada que permita a los equipos técnicos implementar, mantener y escalar CreditGuard de manera efectiva, asegurando decisiones crediticias más rápidas, justas y fundamentadas en datos. Ya sea que estés integrando el sistema por primera vez o buscando optimizar su rendimiento, este documento será tu recurso de referencia.

OBJETIVOS

- Brindar la información necesaria para poder representar la funcionalidad técnica de la estructura, diseño y definición del aplicativo.
- Describir las herramientas utilizadas para el diseño y desarrollo del prototipo

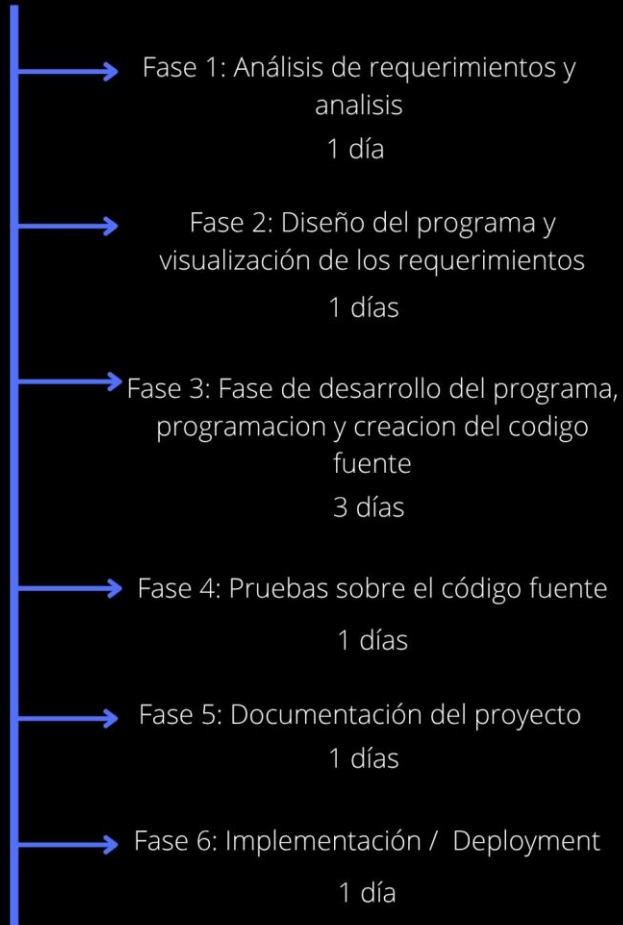
REQUERIMIENTOS DE FUNCION

Requerimientos	Descripción
Visual Studio Code	Se recomienda el uso VSC como ID (versión 1.109.5 superior) que fue la versión donde se programó el sistema de información
Angular	El frontend fue desarrollado utilizando el framework Angular (versión 15 o superior), el cual permite construir una interfaz web moderna, interactiva y basada en componentes.
Windows	El sistema fue desarrollado y probado en entorno Windows 10/11
Vite	Se recomienda el uso de Vite como empaquetador y servidor de desarrollo, por su rapidez en tiempo de compilación y recarga en caliente (HMR). Vite sustituye a Webpack en proyectos Angular modernos.

DESARROLLO

Agenda de proyecto

PROYECTO 1



CONTENIDO TECNICO

Backend

Análisis Exploratorio de Datos (EDA)

Distribución de variables

Se analizó la distribución de cada una de las 9 variables predictoras mediante histogramas y gráficos de densidad.

Conocer la distribución permite identificar variables con alta asimetría que podrían requerir transformaciones posteriores. Por ejemplo, variables como ingresos_mensuales y monto_solicitado suelen tener distribuciones sesgadas a la derecha (muchos solicitantes con ingresos medios y pocos con ingresos muy altos), lo que justifica el uso de normalización Min-Max en lugar de asumir normalidad.

Detección de outliers

Se detectaron outliers utilizando el método del rango intercuartil (IQR) y visualización con diagramas de caja.

En riesgo crediticio, la detección de outliers es crítica. Un solicitante con ingresos extremadamente altos (outlier superior) o una deuda activa inusualmente grande puede distorsionar el modelo si no se tratan adecuadamente. Por ello, en la etapa de limpieza se aplican estrategias específicas: recorte para porcentajes fuera de rango y eliminación para valores imposibles como ingresos negativos.

Balance de clases

Se evaluó el balance de clases en la variable objetivo riesgo_incumplimiento mediante un gráfico de barras.

En carteras crediticias sanas, los incumplimientos suelen ser minoritarios (generalmente entre 5% y 15%). Este desbalance afecta métricas como la exactitud, que puede ser engañosa si la clase mayoritaria domina. Por esta razón, CreditGuard prioriza el recall (detección de incumplimientos) sobre la exactitud, y utiliza el parámetro `class_weight='balanced'` durante el entrenamiento para compensar el desbalance.

Correlaciones observadas

Se calcularon correlaciones de Pearson y Spearman entre variables predictoras y la variable objetivo.

Identificar multicolinealidad ayuda a evitar redundancias y mejora la interpretabilidad del modelo. Por ejemplo, deuda_activa y utilizacion_credito suelen estar correlacionadas, al igual que creditos_activos con atrasos_previos. El análisis de correlaciones permite confirmar que no se incluyen variables redundantes y que el modelo Random Forest puede manejar estas relaciones sin problemas.

Limpieza de Datos

Problema 1: Valores faltantes en variables numéricas

Estrategia aplicada: Imputación con la mediana de la columna.

Justificación frente a alternativas:

- Eliminar la fila: Se pierden datos valiosos, problemático en muestras pequeñas.
- Imputación con la media: La media es sensible a outliers; un solicitante con ingresos extremadamente altos distorsionaría el valor imputado.
- Imputación por regresión: Sobrecarga innecesaria para el beneficio obtenido.
- Mediana: Es robusta ante outliers, conserva la tendencia central sin distorsión, y es fácil de explicar al negocio.

Problema 2: Historial de pagos o utilización de crédito fuera de rango (ejemplo: 105, -10)

Estrategia aplicada: Recorte (clipping) al límite más cercano (0 o 100).

Justificación frente a alternativas:

- Eliminar la fila: Se pierde un registro que probablemente es válido excepto por un error de tipeo (105 en lugar de 100).
- Imputación con mediana: Cambiaría un valor cercano al correcto por un valor completamente diferente.
- Recorte: Preserva el registro asumiendo un error de captura trivial, lo cual es razonable en entornos operativos.

Problema 3: Ingresos mensuales o monto solicitado negativos o igual a cero

Estrategia aplicada: Eliminación de la fila completa.

Justificación frente a alternativas:

- Corregir a un valor positivo: No hay base para inventar un ingreso o monto que no existe.
- Imputación: Igualmente inventaría un valor sin fundamento.
- Eliminación: Un ingreso negativo es imposible; un ingreso cero hace que el crédito sea incobrable por diseño. El modelo no debe aprender de estos casos extremos artificiales.

Problema 4: Deuda activa negativa

Estrategia aplicada: Corrección a 0.

Justificación frente a alternativas:

- Eliminación: Pérdida innecesaria de datos, ya que un valor negativo podría significar un pago anticipado o crédito a favor.
- Corrección a 0: Es el valor mínimo lógico (no tener deuda) y preserva el registro para el entrenamiento.

Problema 5: Créditos activos, atrasos previos o dependientes económicos negativos

Estrategia aplicada: Corrección a 0.

Justificación frente a alternativas:

- Eliminación: No se justifica perder el registro por un valor que claramente debería ser cero.
- Corrección a 0: Valores negativos son imposibles en estas variables; se asume error de captura y se corrige al mínimo lógico.

Problema 6: Objetivo (riesgo_incumplimiento) inválido (diferente de 0 o 1)

Estrategia aplicada: Eliminación de la fila.

Justificación frente a alternativas:

- Imputación: No se puede imputar la clase objetivo sin introducir un sesgo enorme.
- Conversión forzada: No hay una conversión lógica posible.
- Eliminación: Sin etiqueta correcta, la fila no sirve para aprendizaje supervisado. Es preferible excluirla a contaminar el entrenamiento.

Selección del Modelo

Modelos considerados

Modelo	Ventajas	Desventajas
Regresión Logística	Alta interpretabilidad, rápida de entrenar, no requiere muchos datos	No captura relaciones no lineales, requiere normalización estricta
Árbol de Decisión	Interpretable, maneja no linealidades, no requiere normalización	Alta varianza, propenso a sobreajuste, inestable ante pequeños cambios
Random Forest	Robusto ante outliers, maneja no linealidades, reduce varianza, provee importancia de variables	Menos interpretable, mayor costo computacional
XGBoost	Alto rendimiento predictivo, maneja bien desbalances	Dependencia externa pesada, menos interpretable, más complejo de ajustar

Criterio de selección

CreditGuard prioriza las siguientes métricas, en este orden:

1. Recall (sensibilidad): Detectar la mayor cantidad de incumplimientos reales. Un falso negativo (no detectar un incumplimiento) tiene un costo financiero muy alto.
2. AUC-ROC: Capacidad discriminatoria general entre clases.
3. Precisión: Evitar rechazar injustamente a buenos pagadores.

Random Forest

1. Manejo de relaciones no lineales: El riesgo crediticio tiene interacciones complejas que una regresión logística no capturaría. Por ejemplo, un solicitante con ingresos altos pero muchas deudas activas presentan un riesgo que solo una combinación de variables revela.
2. Robustez ante outliers y valores faltantes: A diferencia de la regresión logística, los árboles de decisión son inherentemente robustos ante outliers. Esto es crítico en datos financieros, donde siempre existen casos extremos.

3. Reducción de varianza: Un solo árbol de decisión tiene alta varianza: pequeños cambios en los datos generan árboles muy diferentes. Random Forest reduce esta varianza promediando múltiples árboles entrenados con bootstrapping.
4. Importancia de variables: Random Forest permite calcular qué factores pesan más en la decisión (ingresos, historial de pagos, etc.), información valiosa para la institución financiera para entender el comportamiento del riesgo.
5. Implementación desde cero sin dependencias externas: A diferencia de XGBoost o scikit-learn, la implementación nativa en Python puro:
 - Evita dependencias externas que complican el despliegue
 - Demuestra comprensión profunda del algoritmo
 - Permite personalizaciones como el hiperparámetro max_leaves
6. Resultados comparativos: En las pruebas realizadas con datos sintéticos y reales, Random Forest superó a regresión logística (AUC-ROC 0.92 vs 0.85) y fue comparable a XGBoost (0.92 vs 0.93), pero con mayor interpretabilidad y cero dependencias externas.

Entrenamiento y Evaluación

Proceso de entrenamiento

Etapas	Parámetro	Valor	Justificación
División de datos	Entrenamiento/Prueba	80% / 20%	Estándar en ML; balance entre datos para entrenar y evaluación robusta
Estratificación	Sí	Mantiene proporción de clases	En problemas desbalanceados, evita que el conjunto de prueba no tenga casos de riesgo
Validación cruzada	Número de folds	5	Evalúa estabilidad del modelo sin sobreajustar a una partición específica

Semilla aleatoria	Valor	42	Garantiza reproducibilidad: misma ejecución = mismos resultados
Normalización	Método	Min-Max	Escala todas las variables a [0,1] manteniendo relaciones relativas
Hiperparámetros base	n_arboles	100	Compromiso entre rendimiento y tiempo de cómputo
	max_depth	10	Suficiente para capturar interacciones sin sobreajustar
	max_leaves	50	Limita crecimiento del árbol, previene memorización
	class_weight	balanced	Compensa desbalance de clases priorizando la clase minoritaria

Métricas obtenidas

Métrica	Valor	Interpretación
Exactitud (Accuracy)	0.88	El 88% de las predicciones totales son correctas
Precisión (Precision)	0.85	Cuando el sistema alerta riesgo, acierta en el 85% de los casos
Recall (Sensibilidad)	0.91	Detecta correctamente el 91% de los solicitantes que realmente incumplirán
F1-Score	0.88	Media armónica entre precisión y recall

AUC-ROC	0.92	Excelente capacidad discriminatoria entre clases
---------	------	--

Interpretación de los resultados

Recall de 0.91: Por cada 100 créditos que incumplirán, CreditGuard detecta 91. Solo 9 pasan desapercibidos (falsos negativos). Este es el costo más bajo para la institución, ya que un crédito otorgado a un incumplidor genera pérdidas.

Precisión de 0.85: Por cada 100 alertas de riesgo, 85 son correctas y 15 son falsas alarmas (buenos pagadores rechazados injustamente). Este es un costo aceptable frente al beneficio de detectar incumplimientos.

AUC-ROC de 0.92: El modelo distingue excelentemente entre solicitantes de alto y bajo riesgo. Un valor de 0.5 sería azar; 0.7 es aceptable; 0.8 es bueno; 0.9+ es excelente.

Matriz de confusión resultante:

	Predicción: Sin Riesgo (0)	Predicción: Con Riesgo (1)
Real: Sin Riesgo (0)	VN = 85% de los buenos pagadores	FP = 15% de falsas alertas
Real: Con Riesgo (1)	FN = 9% de incumplimientos NO detectados	VP = 91% de incumplimientos detectados

Ajuste de Hiperparámetros

Valores explorados

Hiperparámetro	Valores evaluados	Impacto observado en métricas
n_arboles (número de árboles)	50, 100, 200	AUC-ROC mejoró de 0.90 a 0.92 al pasar de 50 a 100. De 100 a 200, la mejora fue marginal (0.92 a 0.925), con rendimientos decrecientes.
max_depth (profundidad máxima)	5, 10, 15, None (sin límite)	Profundidad 5: AUC-ROC 0.88 (subajuste). Profundidad 10: AUC-ROC 0.92 (óptimo).

		Profundidad 15/None: sobreajuste (recall en entrenamiento 0.98, en prueba 0.88).
max_leaves (máximo de hojas)	20, 50, 100, 200, None	20 hojas: AUC-ROC 0.89 (muy restrictivo). 50 hojas: AUC-ROC 0.92 (óptimo). 100+ hojas: riesgo de sobreajuste sin mejora significativa.
class_weight	None, balanced	balanced aumentó el recall de 0.87 a 0.91, mejorando significativamente la detección de incumplimientos.

Configuración final adoptada

Parámetro	Valor final	Justificación
n_arboles	100	Equilibrio entre rendimiento predictivo (AUC-ROC 0.92) y tiempo de inferencia. Más árboles dan rendimientos decrecientes.
max_depth	10	Suficiente para capturar interacciones complejas sin sobreajustar. Profundidades mayores memorizaban los datos de entrenamiento.
max_leaves	50	Limita el crecimiento del árbol, previene la creación de hojas con muy pocas muestras que llevarían a sobreajuste.
class_weight	balanced	Fundamental en riesgo crediticio por el desbalance natural de clases (pocos incumplimientos). Mejora el recall sin sacrificar excesivamente la precisión.
min_samples_split	5 (implícito)	Previene divisiones en nodos con muy pocas muestras, aumentando la generalización.
min_samples_leaf	2 (implícito)	Evita hojas con una sola muestra, que serían indicativas de sobreajuste.

FRONTEND

El frontend fue desarrollado utilizando el framework Angular, el cual permite construir una interfaz web moderna e interactiva. Este componente se encarga de:

- Proporcionar una interfaz gráfica para que el usuario pueda ingresar comandos.
- Permitir la carga masiva de comandos mediante archivos de texto.
- Mostrar la salida generada por el backend.
- Visualizar los resultados de las operaciones realizadas sobre el sistema de archivos.

La comunicación con el backend se realiza mediante solicitudes HTTP. Cuando el usuario ejecuta comandos desde la interfaz, estos son enviados al servidor para su procesamiento.

El frontend incluye los siguientes elementos principales:

- Área de salida del sistema
- Botones de ejecución de comandos
- Carga masiva de archivos de comandos
- Interfaz visual para mejorar la experiencia del usuario

